Rain BitTorrent 클라이언트 데이터 흐름 분석

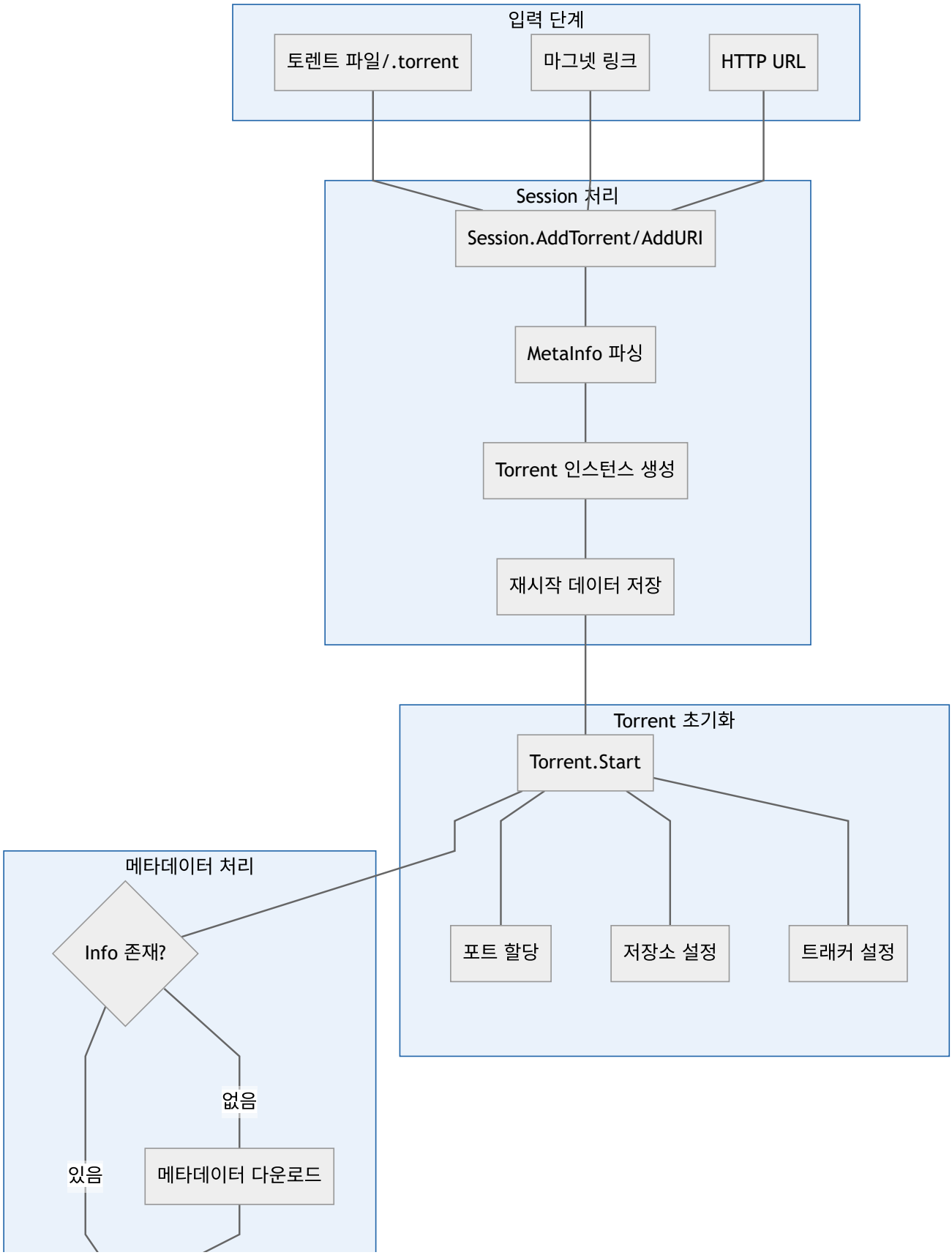
목차

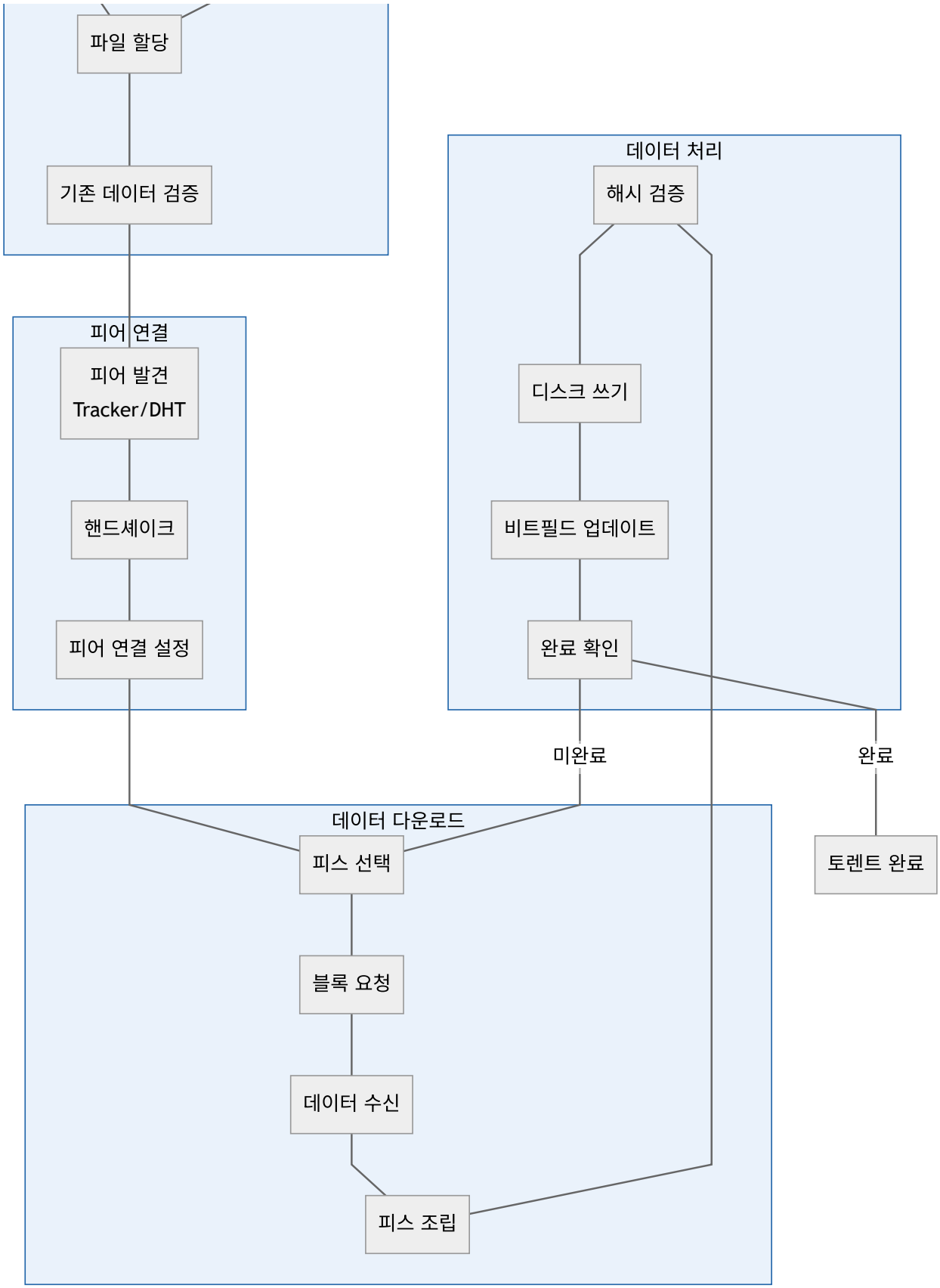
1. 개요
2. 전체 데이터 흐름 개요
3. 토렌트 추가 프로세스
4. 메타데이터 다운로드 흐름
5. 피어 연결 및 핸드셰이크 흐름
6. 피스 다운로드 흐름
7. 데이터 검증 및 저장 흐름
8. 컴포넌트 간 상호작용
9. 주요 인터페이스와 데이터 구조

개요

이 문서는 Rain BitTorrent 클라이언트에서 토렌트 추가부터 완료까지의 전체 데이터 흐름을 분석합니다. 각 단계별로 데이터가 어떻게 처리되고 컴포넌트 간에 어떻게 전달되는지를 상세히 설명합니다.

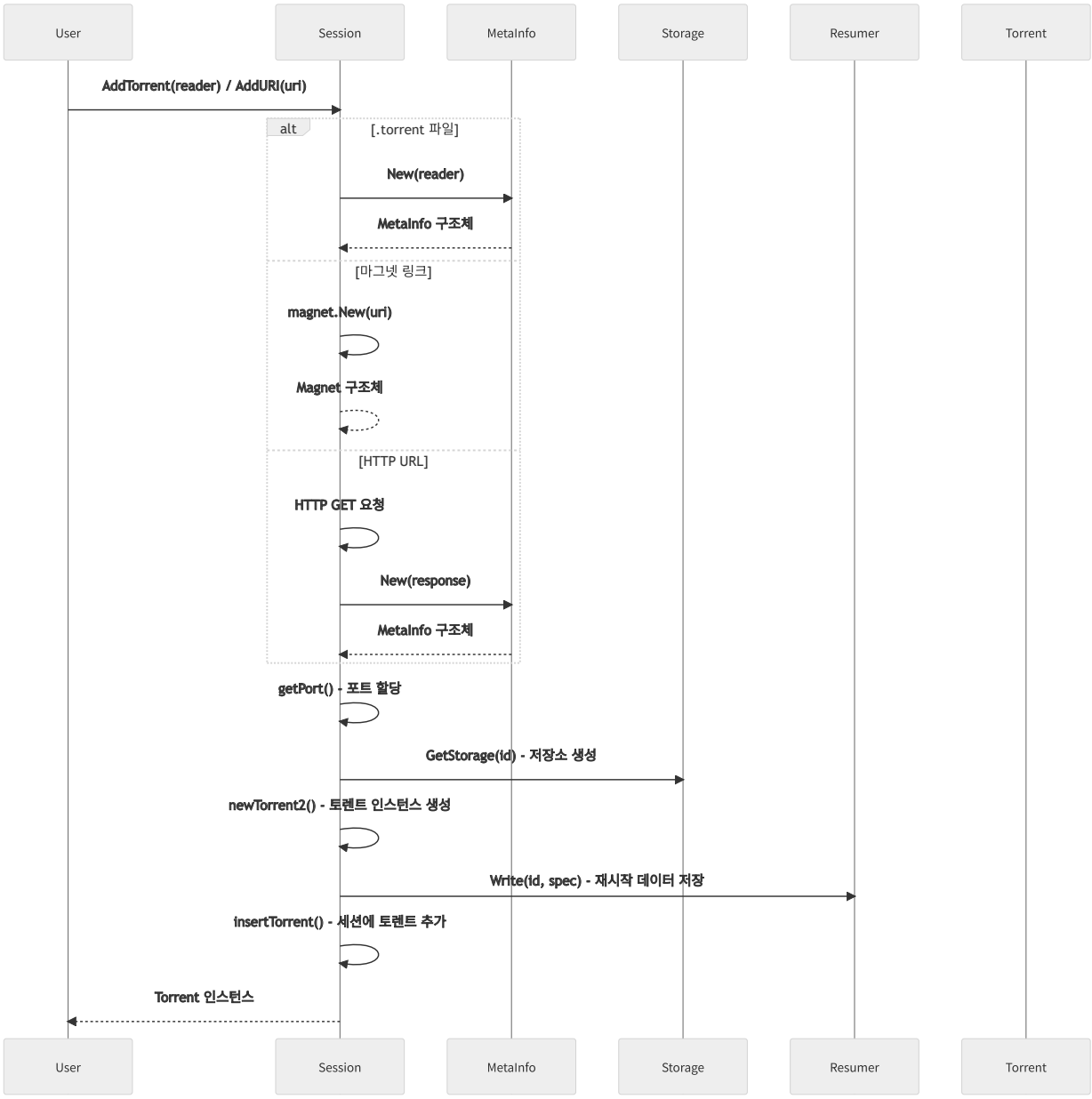
전체 데이터 흐름 개요


입력 단계
토렌트 파일/.torrent
마그넷 링크
HTTP URL
Session 처리
Session.AddTorrent/AddURI
MetaInfo 파싱
Torrent 인스턴스 생성
재시작 데이터 저장
Torrent 초기화
Torrent.Start
포트 할당
저장소 설정
트래커 설정
메타데이터 처리
Info 존재?
없음
있음
메타데이터 다운로드


파일 할당
기존 데이터 검증
데이터 처리
해시 검증
피어 연결
피어 발견
Tracker/DHT
디스크 쓰기
핸드셰이크
비트필드 업데이트
피어 연결 설정
완료 확인
미완료
완료
데이터 다운로드
피스 선택
토렌트 완료
블록 요청
데이터 수신
피스 조립

토렌트 추가 프로세스

1. 입력 처리 (`Session.AddTorrent/AddURI`)


User
Session
MetaInfo
Storage
Resumer
Torrent
AddTorrent(reader) / AddURI(uri)
alt
[.torrent 파일]
New(reader)
MetaInfo 구조체
[마그넷 링크]
magnet.New(uri)
Magnet 구조체
[HTTP URL]
HTTP GET 요청
New(response)
MetaInfo 구조체
getPort() - 포트 할당
GetStorage(id) - 저장소 생성
newTorrent2() - 토렌트 인스턴스 생성
Write(id, spec) - 재시작 데이터 저장
insertTorrent() - 세션에 토렌트 추가
Torrent 인스턴스
User
Session
MetaInfo
Storage
Resumer
Torrent

2. 토렌트 인스턴스 생성 과정

핵심 함수: `newTorrent2()`

```
func newTorrent2(
s *Session,
id string,
addedAt time.Time,
infoHash []byte,
sto storage.Storage,
name string,
port int,
trackers []tracker.Tracker,
fixedPeers []string,
info *metainfo.Info,
bf *bitfield.Bitfield,
stats resumer.Stats,
ws []*webseedsource.WebseedSource,
stopAfterDownload bool,
```

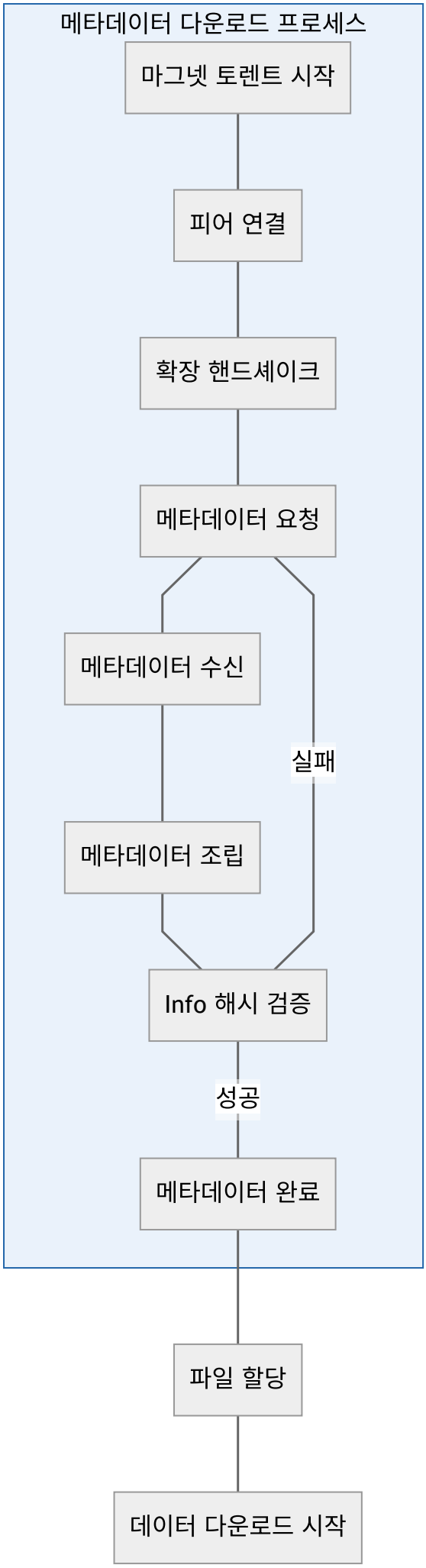
```
stopAfterMetadata  bool,

completeCmdRun  bool,

) (*torrent, error)
```

생성되는 주요 구조체:

- 채널들: `peerDisconnectedC`, `messages`, `pieceMessagesC`
- 맵들: `peers`, `pieceDownloaders`, `infoDownloaders`
- 컴포넌트들: `unchoker`, `addrList`, `piecePool`

메타데이터 다운로드 흐름

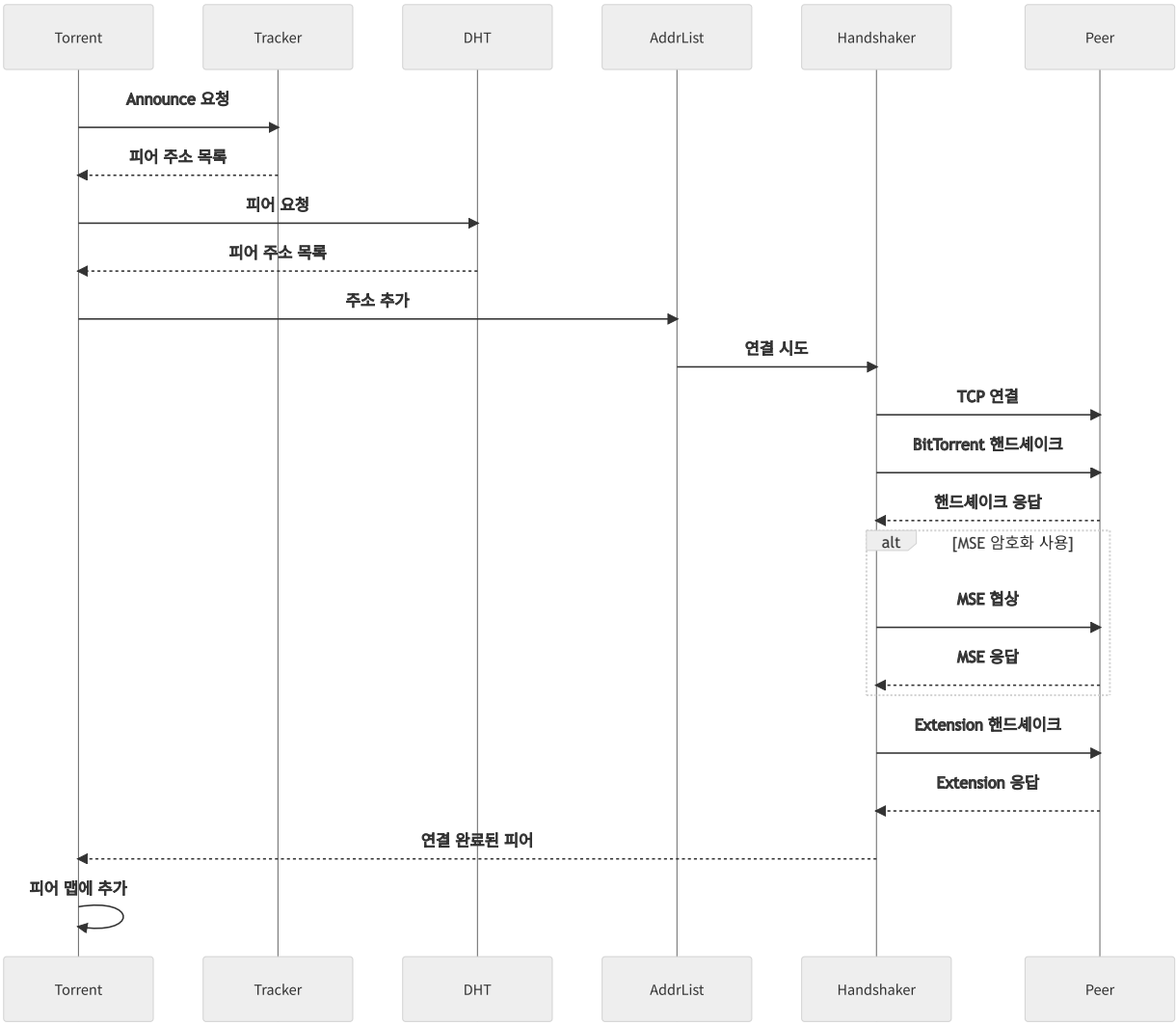
마그넷 링크의 경우 메타데이터(Info)가 없으므로 별도 다운로드가 필요합니다.


메타데이터 다운로드 프로세스
마그넷 토렌트 시작
피어 연결
확장 핸드셰이크
메타데이터 요청
메타데이터 수신
실패
메타데이터 조립
Info 해시 검증
성공
메타데이터 완료
파일 할당
데이터 다운로드 시작

관련 컴포넌트:

- `infodownloader.InfoDownloader` : 메타데이터 다운로드 관리
- `metainfo.Info` : 토렌트 메타데이터 구조체
- Extension Protocol (BEP 9): 메타데이터 교환 프로토콜

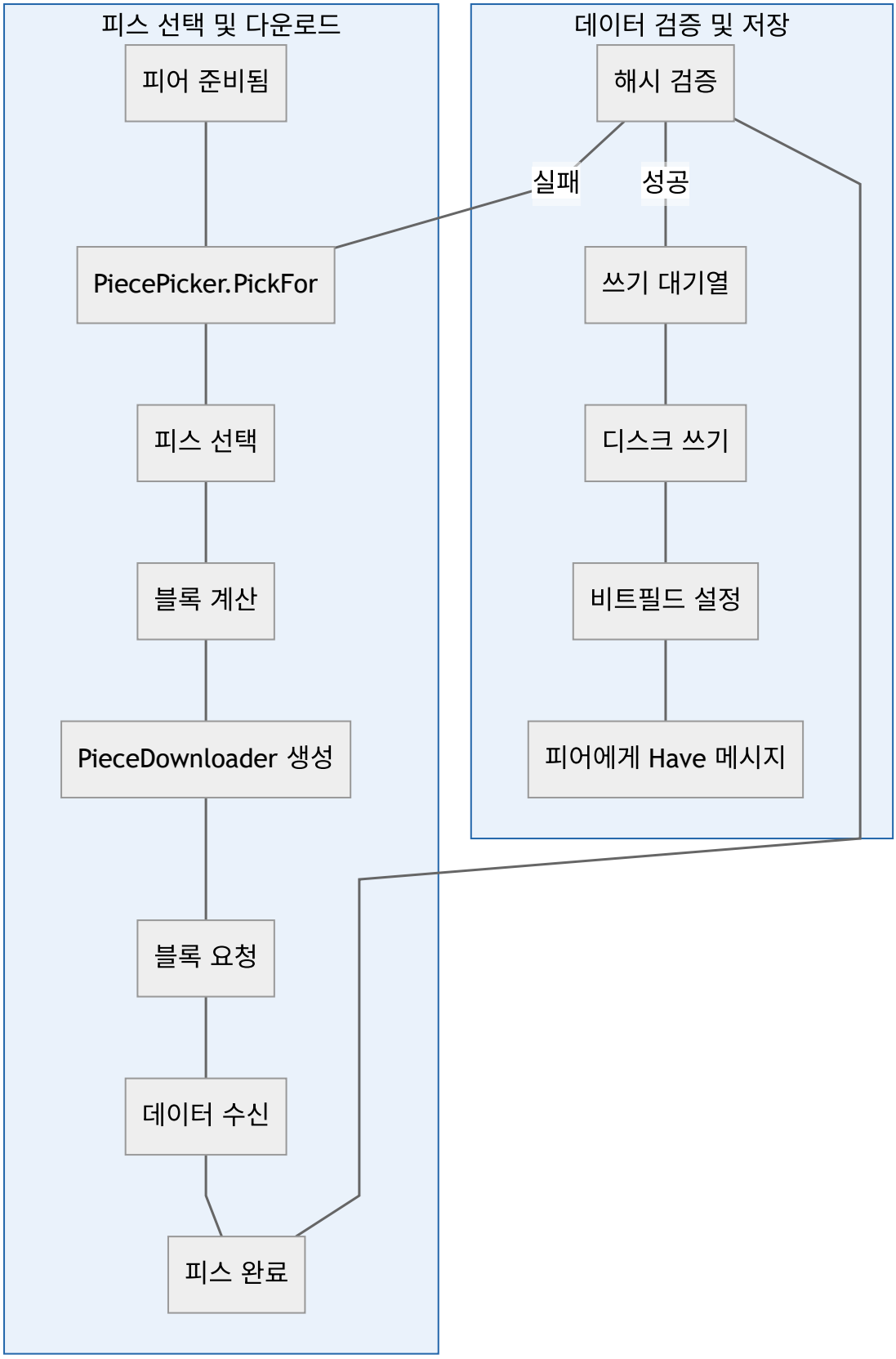
피어 연결 및 핸드셰이크 흐름


Torrent
Tracker
DHT
AddrList
Handshaker
Peer
Announce 요청
피어 주소 목록
피어 요청
피어 주소 목록
주소 추가
연결 시도
TCP 연결
BitTorrent 핸드셰이크
핸드셰이크 응답
alt
[MSE 암호화 사용]
MSE 협상
MSE 응답
Extension 핸드셰이크
Extension 응답
연결 완료된 피어
피어 맵에 추가
Torrent
Tracker
DHT
AddrList
Handshaker
Peer

핸드셰이크 단계:

1. **TCP 연결**: 기본 네트워크 연결 설정
2. **BitTorrent 핸드셰이크**: 프로토콜 식별 및 InfoHash 확인
3. **MSE 협상** (선택적): 메시지 스트림 암호화
4. **Extension 핸드셰이크**: 확장 프로토콜 지원 협상

피스 다운로드 흐름



피스 선택 알고리즘

PiecePicker의 전략:

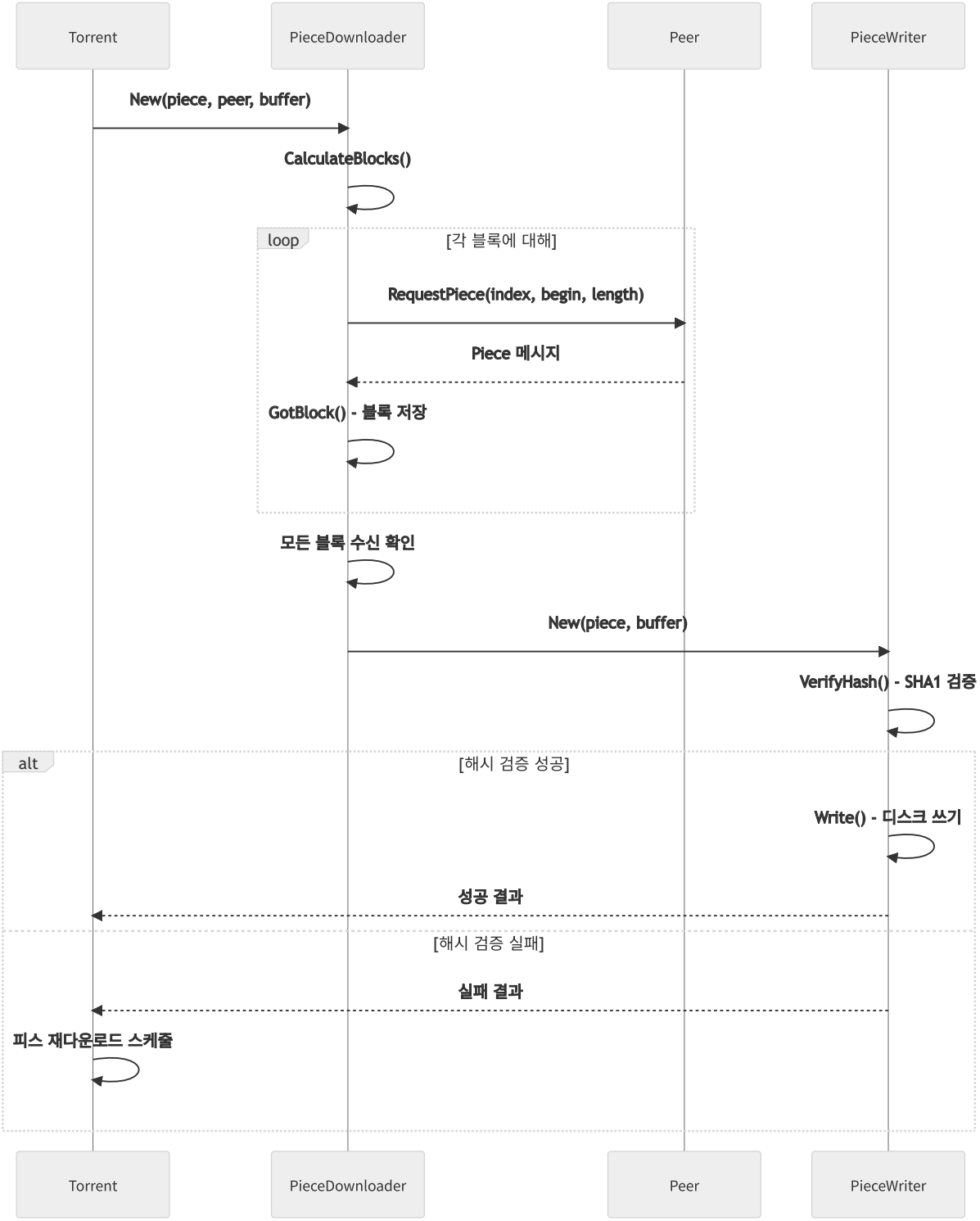
1. **Random First Piece**: 첫 번째 피스는 랜덤 선택

2. **Rarest First**: 가장 희귀한 피스 우선 선택

3. **Endgame Mode**: 마지막 몇 개 피스는 중복 다운로드

```
type PiecePicker struct {

pieces []myPiece

piecesByAvailability []*myPiece

maxDuplicateDownload  int

endgame  bool

}
```

블록 다운로드 과정


Torrent
PieceDownloader
Peer
PieceWriter
New(piece, peer, buffer)
CalculateBlocks()
loop
[각 블록에 대해]
RequestPiece(index, begin, length)
Piece 메시지
GotBlock() - 블록 저장
모든 블록 수신 확인
New(piece, buffer)
VerifyHash() - SHA1 검증
alt
[해시 검증 성공]
Write() - 디스크 쓰기
성공 결과
[해시 검증 실패]
실패 결과
피스 재다운로드 스케줄
Torrent
PieceDownloader
Peer
PieceWriter

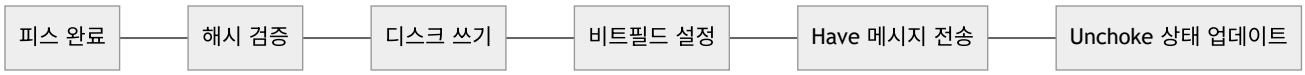
데이터 검증 및 저장 흐름

1. 해시 검증 과정

```
func (w *PieceWriter) Run(resultC  chan *PieceWriter, closeC  chan  struct{},
writesPerSecond, writeBytesPerSecond metrics.Meter,
sem *semaphore.Semaphore) {
// SHA1 해시 검증
w.HashOK = w.Piece.VerifyHash(w.Buffer.Data, sha1.New())
if  w.HashOK {
// 메트릭 업데이트
writesPerSecond.Mark(1)
writeBytesPerSecond.Mark(int64(len(w.Buffer.Data)))
// 세마포어로 동시 쓰기 제한
sem.Wait()
_, w.Error = w.Piece.Data.Write(w.Buffer.Data)
sem.Signal()
}
// 결과 전송
select {
```

```
case  resultC <- w:
case <-closeC:
}
}
```

2. 비트필드 업데이트



컴포넌트 간 상호작용

1. 토렌트 메인 루프 (`torrent.run()`)

토렌트의 핵심은 이벤트 기반 메인 루프입니다:

```
func (t *torrent) run() {
for {
select {
case <-t.closeC:
// 토렌트 종료
case <-t.startCommandC:
t.start()
case <-t.stopCommandC:
t.stop(nil)
```

```
case addrs := <-t.addrsFromTrackers:
t.handleNewPeers(addrs, peersource.Tracker)
case addrs := <-t.dhtPeersC:
t.handleNewPeers(addrs, peersource.DHT)
case pw := <-t.pieceWriterResultC:
t.handlePieceWriteDone(pw)
case pm := <-t.pieceMessagesC.ReceiveC():
t.handlePieceMessage(pm)
// ... 기타 이벤트들
}
}
}
```

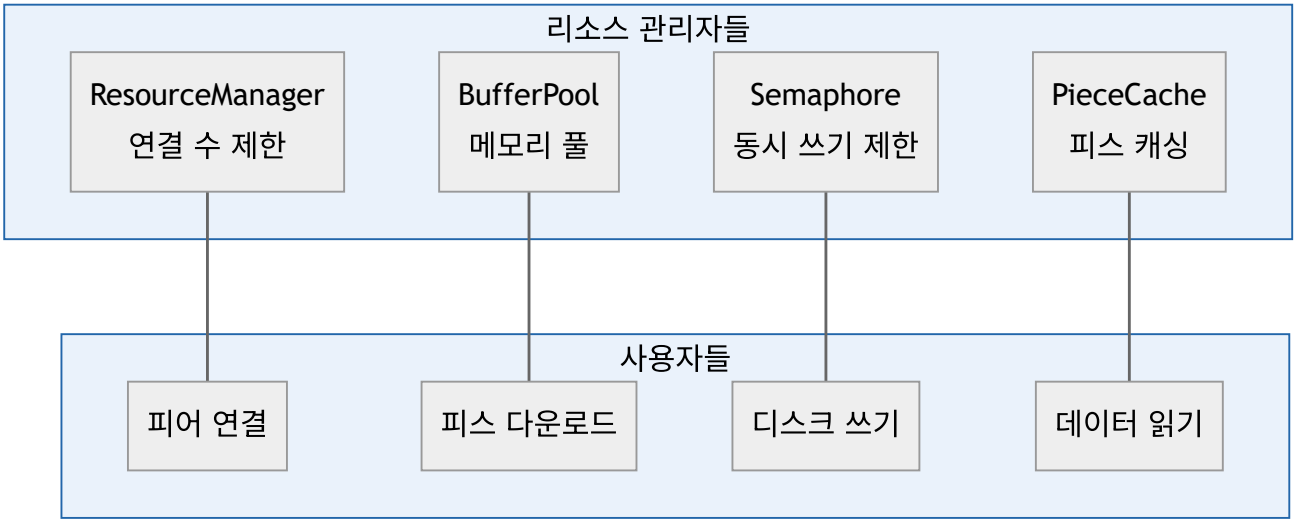
2. 채널 기반 통신

Rain은 Go의 채널을 활용한 비동기 통신을 사용합니다:

주요 채널들:

- `addrsFromTrackers` : 트래커로부터 피어 주소 수신
- `pieceMessagesC` : 피어로부터 피스 메시지 수신
- `pieceWriterResultC` : 피스 쓰기 완료 알림
- `peerDisconnectedC` : 피어 연결 해제 알림

3. 리소스 관리



주요 인터페이스와 데이터 구조

1. 핵심 인터페이스

```
// Storage 인터페이스 - 파일 저장 추상화
type Storage interface {
Open(name  string, size  int64) (File, bool, error)
}

// Tracker 인터페이스 - 트래커 통신 추상화
type Tracker interface {
Announce(ctx context.Context, req AnnounceRequest) (*AnnounceResponse, error)
URL() string
}

// Peer 인터페이스 - 피어 통신 추상화
```

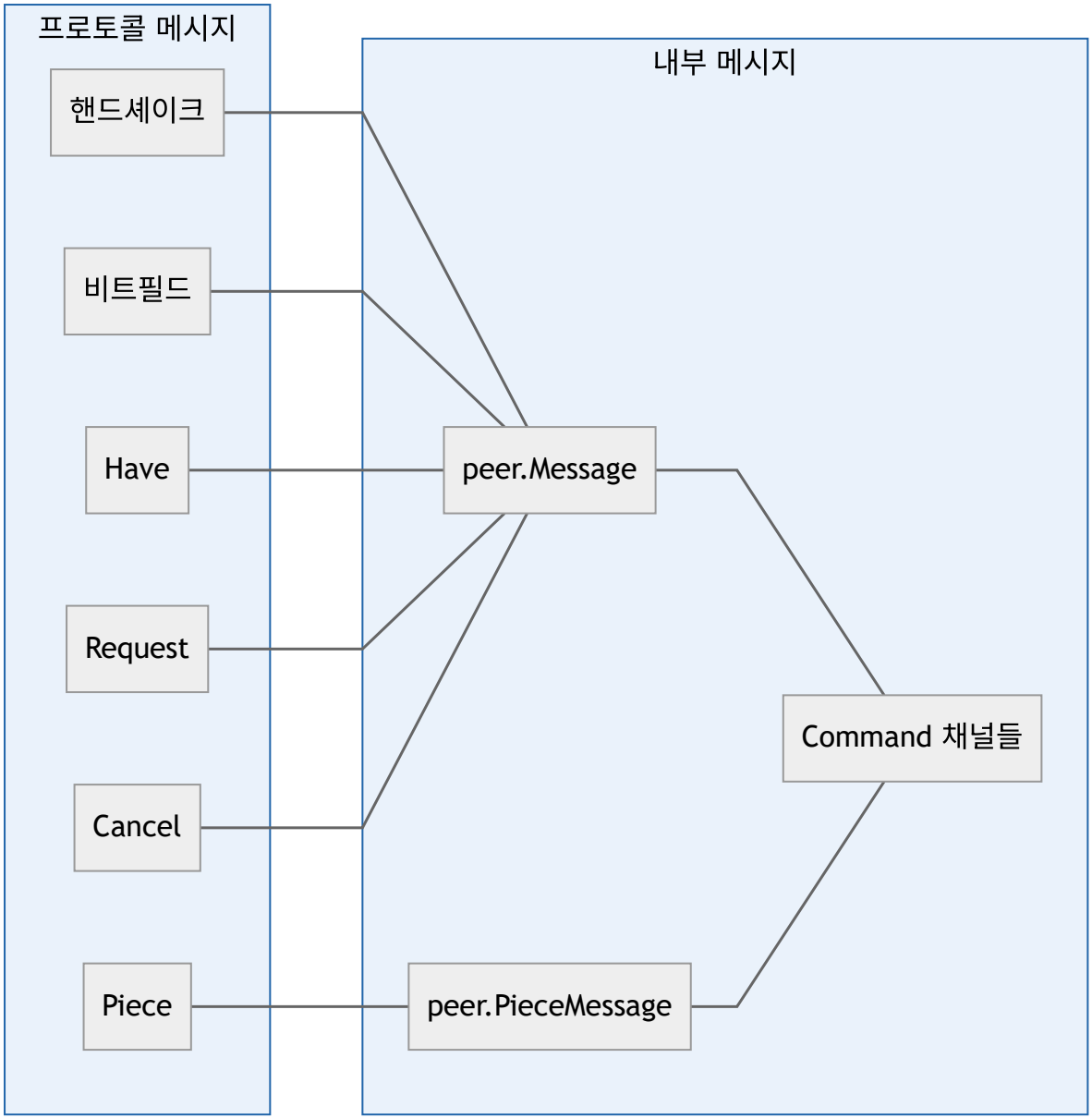
```go
type Peer interface {
RequestPiece(index, begin, length  uint32)
CancelPiece(index, begin, length  uint32)
EnabledFast() bool
}
```

2. 핵심 데이터 구조

```go
// 피스 정보
type Piece struct {
Index  uint32
Hash [20]byte
Length  uint32
Data FilePiece
Done  bool
}

// 비트필드 - 피스 보유 상태
type Bitfield struct {
b []byte
n  uint32
}
```

```go
// 피어 상태
type Peer struct {
*peerconn.Conn
ID [20]byte
Bitfield *bitfield.Bitfield
ClientInterested  bool
ClientChoking  bool
PeerInterested  bool
PeerChoking  bool
Downloading  bool
}
```

3. 메시지 흐름



이러한 데이터 흐름을 통해 Rain BitTorrent 클라이언트는 효율적이고 안정적인 파일 공유를 실현합니다. 각 컴포넌트는 명확한 책임을 가지고 있으며, 채널 기반의 비동기 통신을 통해 높은 성능과 확장성을 제공합니다.

관련 문서

선행 지식

- 01-system-architecture.md - 전체 시스템 아키텍처 이해 필수

관련 주제

- 04-01-networking-packages-analysis.md - 네트워크 메시지 처리 구현

- 06-02-concurrency-analysis.md - 채널 기반 메시지 전달 패턴

후속 학습

- 03-01-torrent-lifecycle-process.md - 토렌트 생명주기 상세 과정
- 05-01-dht-implementation-analysis.md - DHT 데이터 흐름
- 05-04-webseed-support-analysis.md - WebSeed 데이터 흐름

> **네비게이션:** 이전: 시스템 아키텍처 | 목차 | 다음: 토렌트 라이프사이클